An Empirical Comparison of Scheduling Policies in a Distributed Task Queue

Latency, deadline compliance, fairness and starvation under controlled load

Anish Choudhury

September 2026 · interactive results: anishc23.github.io/distributed-task-queue-scheduler

Contents

Abstract 1
1. Introduction 2
2. System design 3
3. Methodology 5
4. Results 6
5. Threats to validity 26
6. Conclusions 29
Appendix A: Reproducing these results 31
Appendix B: Experiment inventory 32

Abstract

Most distributed task queues hard-code a single queue discipline, almost always arrival order. This report treats the discipline as an experimental variable. A task queue was built in Go on Redis Streams in which the scheduling policy is a pluggable component chosen by configuration, and four policies — FIFO, strict priority, earliest deadline first (EDF) and weighted fair queuing (WFQ) — were compared across four synthetic workloads, seven levels of offered load, and two models of task execution.

The study comprises **1,567 experiments over 5.85 million tasks**. Four results are reported. First, scheduling policy is decisive only in a narrow band around service capacity: the spread between the best and worst policy's 99th-percentile latency is 0.1% at half capacity, peaks at 93% at capacity, and falls to 1.1% at three times capacity. Second, centralising the scheduling decision costs approximately 1.5% of throughput, while *using* that decision costs 6–10% and returns improvements of one to two orders of magnitude in the metric each policy targets. Third, and least expected, the common practice of simulating task execution with sleeps is not a neutral simplification: it systematically flatters deadline-aware scheduling, because EDF's advantage depends on declared task durations that real execution invalidates. WFQ is unaffected by the same error, and the asymmetry has a clean explanation. Fourth, the availability cost of that central scheduler is bounded and measurable: enforcing its single-writer invariant with a fenced Redis lease costs under 1% of scheduling throughput, a crash costs a median 4.4 s of dispatch outage against a 5 s

lease, and with no standby the queue never recovers at all. That last result is tested against the failure it is actually designed for — a leader frozen past its lease and then resumed, rather than killed — where the fence, and not the lease, is what refused the returning leader in 11 of 15 trials. Pushed one level further, the fence turns out to inherit the durability of the store beneath it: a Redis failover to a lagging replica issued one fencing token to two leaders in 10 trials out of 10, and the fix is a store setting rather than a scheduler change. Cut off from a Redis that stays up, rather than stopped, a leader kept its role through any partition shorter than its lease, lost it to a standby after one longer, and was refused by the fence on its return in 11 of 12 trials. Across seven partition lengths the outage grew with the partition only until the lease ran out, and stayed inside a crash's window from then on. Measured against Asynq, an external queue with no central scheduler, the architecture costs about 30% of throughput on tasks shorter than three milliseconds and nothing measurable on tasks of five milliseconds or more.

Three quantitative claims made earlier in this work were subsequently found to be wrong and are corrected here. That process is reported rather than concealed, because the corrections are among the more instructive outcomes.

1. Introduction

1.1 Research question

> How does the choice of scheduling policy affect latency, throughput, deadline compliance, fairness and starvation in a distributed task queue under different workload distributions and levels of offered load?

The contribution is empirical rather than algorithmic. All four policies are well established. What is measured is their behaviour under controlled conditions in one concrete system, together with the conditions under which the differences between them matter at all.

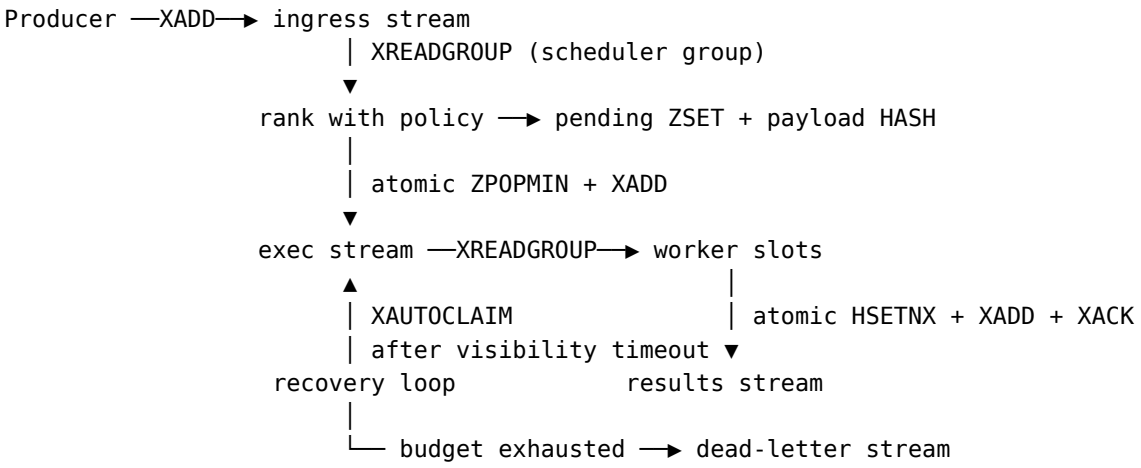
1.2 Scope and honest framing

The system is a task queue built on a single Redis instance with a single-writer scheduler. Most of the genuinely hard distributed-systems problems — consensus, partitioning, network partitions, clock skew — are absent or delegated to Redis. It is more accurately described as a scheduling study on a Redis-backed queue than as a distributed system, and is described that way throughout.

One of them is not delegated. The single-writer scheduler is a coordination problem in its own right, and Section 4.7 treats it as one: leadership is a Redis lease with a monotonic fencing token, failure detection is lease expiry, and the resulting failover behaviour is measured rather than asserted. The fencing argument is the standard one — a lock alone cannot stop a process that was paused past its lease, so the check has to live at the resource — and implementing the fence rather than only the lock is what makes "exactly one scheduler dispatches" a property of the system rather than of its deployment manifest. That is a narrow slice of the distributed-systems problem space, and it is claimed as nothing more.

2. System design

2.1 Data flow



Producers write only to the ingress stream; workers see only what the scheduler has already ranked and dispatched. `max_in_flight` bounds dispatched-but- unfinished work so the backlog remains in the scheduler's pending set, where the policy controls it, rather than in the execution stream, where it would not.

2.2 The scheduling abstraction

A policy converts a task into an ordering key and nothing else:

```
type Key struct {
    Score  float64 // lower dispatches first
    Member string  // "<zero-padded submit millis>|<task id>"
}
```

The surrounding machinery always dispatches the smallest key. This exploits a property of Redis: sorted sets order equal-score members lexicographically by byte value, which is exactly Go string comparison. The consequence is that the in-memory priority queue used by unit tests and the `ZPOPMIN` used in production **cannot disagree**, so the ordering asserted in tests is the ordering the deployed system produces. An integration test verifies the equivalence against a live Redis.

2.3 Policies

FIFO orders by submission time. The experimental baseline.

Strict priority packs priority and submission time into one float64 as `(255 − priority)·10¹³ + submitted_millis`. Unix milliseconds stay below 10^{13} until the year 2286, so a timestamp cannot leak into an adjacent priority band, and the largest score remains inside the range where float64 represents integers exactly. No ageing: starvation is measured, not mitigated.

EDF orders by absolute deadline, with tasks lacking deadlines sorted last.

WFQ implements self-clocked fair queuing over tenants. On admission of task t from tenant k with weight w_k:

```
start  = max(virtualTime, lastFinish[k])
finish = start + cost(t) / w_k
```

and on dispatch, `virtualTime = max(virtualTime, finish)`. Because a newly active tenant starts at `max(virtualTime, lastFinish)`, an idle tenant cannot bank credit and flood the queue. Documented approximations: virtual time advances on dispatch events rather than by emulating a fluid server; scheduling is non-preemptive at whole-task granularity; and `cost(t)` uses the task's declared duration, an oracle assumption that Section 5.3 shows is consequential.

2.4 Correctness

Four Lua scripts carry the system's correctness, each atomic inside Redis. Dispatch performs `ZPOPMIN` and `XADD` in one unit, so a task cannot be lost between selection and delivery. Completion is guarded by `HSETNX` on a hash of completed task IDs, in the same script that writes the result record and acknowledges the delivery.

The guarantee is stated precisely: **at-least-once delivery with completion recorded exactly once per task ID.** A task body may execute more than once; exactly-once delivery is not claimed. This was verified under a genuine race by setting the visibility timeout below the task duration, so the recovery loop reclaims work that is still legitimately running and two workers race to complete the same task. Duplicate completions were suppressed and every task appeared exactly once in the results stream.

A second invariant is enforced in the same place: **exactly one scheduler ranks and dispatches at a time.** This one is not about losing work, it is about losing meaning. The policy's virtual clock lives in the scheduler's memory, so two schedulers each advance their own copy against part of the traffic and then overwrite each other's persisted state. Nothing errors and no counter moves; only the fairness numbers are wrong.

Leadership is a Redis lease (`SET NX PX`, renewed, released on clean shutdown), and each grant carries an **epoch** from a monotonic counter. The epoch, not the lease, is what makes this sound. A leader paused past its TTL by a garbage-collection pause or a suspended container resumes believing it leads, and no amount of careful renewal closes that window because the leader is not executing during it. So the epoch travels with every write the leader makes, and the check runs inside the same atomic script as the write:

```
if epoch < fence then return redis.error_reply('TQFENCED ...') end
if epoch > fence then redis.call('SET', fenceKey, epoch) end
```

Admission and dispatch are both guarded. Guarding admission matters for a reason that is easy to miss: the score written at admission is not a property of the task, it is produced by the policy's virtual clock, so a superseded leader that could still admit would insert tasks at positions derived from a clock it no longer owns. A non-positive epoch means election is disabled and the guard is inert, which is the path the benchmark harness takes.

3. Methodology

3.1 Experimental design

Four schedulers × four workloads, repeated with controlled seeds. Each experiment runs in its own Redis key namespace, and every CSV row repeats the full run identity, so results from different runs cannot be pooled by accident.

Workloads are finite and seeded. For a given seed and configuration the generated task sequence is byte-identical on any machine: every random value comes from one seeded source consumed in a fixed order per task. Timing is not reproducible, only the workload; repetitions and reported variance are the mitigation.

3.2 Offered load

Comparing policies across workloads is only meaningful at equal load relative to what the system can serve:

```
capacity     = worker_slots / mean_execution_seconds
offered_load = mean_arrival_rate / capacity
```

Computing this per workload proved necessary rather than pedantic. The heavy-tailed workload has a mean service time of 51.4 ms against 50.1 ms for the others, so an identical arrival rate is a different load. More seriously, the bursty workload ignores its nominal arrival-rate parameter entirely — its arrivals derive from the burst schedule — and its duration-weighted mean rate was 500/s against 400/s for the other three. **Every bursty measurement in an earlier revision of this work was therefore taken at 1.57× capacity while the others sat at 1.25×**, which is a confound, not a rounding difference, and explains why bursty showed a 0.94 deadline miss rate: it was simply run harder.

The load sweep is valid only if changing the arrival rate changes nothing else. It does not: the generator draws arrival gap, tenant, duration, priority and deadline jitter in a fixed order per task, so altering the rate alters gap values without moving the stream position. A test asserts byte-identical durations, priorities, tenants and deadlines across a ninefold rate change for all four workload types.

3.3 Repetition counts

How many repetitions a workload needs is not uniform. All four were re-run at 25 repetitions to find out, giving a 5-versus-25 comparison over 96 workload/scheduler/metric combinations.

workload	5-rep CIs missing the truth	median shift	verdict
uniform	8 / 24	0.60%	converged
bursty	1 / 24	0.13%	converged
multi_tenant	0 / 24	0.26%	converged
heavy_tailed	1 / 24	**8.42%**	**not converged**

Interval coverage is the wrong convergence diagnostic, and `uniform` proves it. `uniform` is the most stable workload in the matrix and has by far the *most* confidence intervals that missed — eight, against one for the genuinely unconverged heavy-tailed workload. The mechanism is mechanical: a stable metric produces a narrow interval that a 1% shift escapes, while a noisy metric produces a wide interval that almost nothing escapes. Coverage measures how stable a metric is, not how converged the estimate is. The **median shift in the point estimate** separates the workloads cleanly where coverage does not.

The distinguishing property is the task-duration distribution, not the arrival pattern: bursty arrivals and skewed tenants both average out over 4,000 tasks; a Pareto tail does not.

3.4 Environment

macOS 26.6 on an Apple M5 (10 cores), Redis 8.10.1 on localhost, Go 1.27. The main matrix uses 4 worker processes × 4 slots = 16 slots against a capacity of approximately 319 tasks/s. Real-work experiments use 8 slots on 10 cores, for reasons given in Section 5.3.

The failover experiments of Section 4.7 have a stricter requirement, because they measure wall-clock gaps rather than aggregate rates: anything else running on the machine lands directly in the number. Two runs were discarded for that reason and are not reported. The first was invalidated by starting a race-detector test suite alongside it. The second was invalidated by the laptop idle-sleeping partway through an unattended run, which produced two trials reporting seventeen-minute outages against a five-second lease.

The second failure is now caught rather than averaged in. A crash outage is bounded above by the lease TTL by construction, so the harness flags any trial exceeding five times the TTL as suspect and the analysis drops it while printing what it dropped. Reporting a number that cannot come from the mechanism under test is a failure of the harness, not of the run.

4. Results

4.1 Main matrix

Twenty-five repetitions per cell, 400 experiments, 1.6 million tasks. Latency and wait columns are seconds. Arrivals were configured at 400 tasks/s, which is 1.25× capacity for uniform and multi-tenant and about 1.28× for heavy-tailed. **The four bursty rows were not run at that load.** The bursty workload ignores the configured rate (Section 3.2), and these rows were measured at about 1.57× capacity. They are kept because they are what was measured, but they are not load-matched to the other twelve, and the bursty-against-uniform gap in this table is mostly that difference in load. Section 4.4′s sweep, which derives each workload's rate from measured capacity, is the load-controlled comparison.

workload	sched	p50	p99	maxwait	miss	tasks/s	Jain
uniform	fifo	1.486	2.917	2.900	0.791	308.8	0.999
uniform	prio	0.109	4.491	4.492	0.462	308.3	0.999
uniform	edf	1.483	2.992	3.069	0.788	308.3	0.999
uniform	wfq	1.476	3.240	3.270	0.786	308.2	0.999
bursty	fifo	3.216	6.245	6.273	0.943	308.9	0.999
bursty	prio	1.705	7.538	7.561	0.704	308.1	0.999
bursty	edf	3.158	6.306	6.389	0.937	308.1	0.999
bursty	wfq	3.184	6.345	6.389	0.943	308.2	0.999
heavy-tail	fifo	0.749	1.686	1.613	0.482	282.7	0.979
heavy-tail	prio	0.096	2.685	2.681	0.343	281.7	0.979
heavy-tail	edf	0.031	1.271	6.163	**0.001**	257.6	0.979
heavy-tail	wfq	0.394	2.487	2.574	0.354	270.5	0.979
multi-ten	fifo	1.479	2.920	2.907	0.780	308.4	0.246
multi-ten	prio	0.105	4.462	4.467	0.457	308.4	0.246
multi-ten	edf	1.459	2.976	3.044	0.775	308.3	0.246
multi-ten	wfq	1.490	2.932	2.910	0.727	308.4	0.246

Throughput is flat within 0.3% across policies on three of four workloads, as it should be: work-conserving schedulers change who waits, not how much work gets done. The exception is heavy-tailed, where EDF delivers 257.6 tasks/s against FIFO's 282.7 and utilisation falls to 0.73 from 0.80 — deferring long tasks leaves slots idle at the end of a run while the deferred work drains.

4.2 Starvation under strict priority

p95 latency by priority level, uniform workload, 25 repetitions pooled over 100,000 tasks per scheduler:

sched	prio 0	prio 1	prio 2	prio 3	prio 4	spread
fifo	2.796	2.797	2.797	2.797	2.819	1.0×
priority	**4.416**	0.079	0.075	0.074	0.073	**60.4×**
edf	2.816	2.813	2.809	2.811	2.834	1.0×
wfq	2.954	2.953	2.947	2.941	2.967	1.0×

Everything above priority 0 completes in about 75 ms while priority 0 waits 4.4 s. FIFO, EDF and WFQ show a flat 1.0×, because none of them reads the priority field.

4.3 Fairness under tenant skew

Jain's fairness index is defined over completed service seconds per tenant:

$$J(x) = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$$

with zero-service tenants included as zeros — omitting them would report perfect fairness for a queue that served exactly one tenant.

The index cannot distinguish the policies on this workload, and it is important to say why. Under the 90/3/3/2/2 skew every policy reports 0.246, near the 1/n = 0.2 floor, because the index is *demand-limited*: tenants B–E submit a few percent of the work, so no scheduler can grant them a large share of service. That is a property of the workload, not a failure of the scheduler.

Per-tenant latency is the metric that does distinguish them:

sched	A (90%)	small (mean)	ratio
fifo	2.798	2.762	0.987×
priority	4.253	4.191	0.986×
edf	2.791	2.754	0.987×
wfq	2.858	**0.078**	**0.027×**

WFQ gives the small tenants 35× lower p95 latency at a 2% cost to the dominant tenant. Under the other three policies the small tenants are queued behind the noisy neighbour's backlog and see essentially its latency — equal treatment that is not isolation. Reporting the fairness index alone would have shown four identical bars and missed the effect entirely.

4.4 Policy choice depends on load

Seven load levels from 0.5× to 3× capacity, 336 experiments, 1.34 million tasks. Spread between best and worst policy p99 on the uniform workload:

offered load	0.50×	0.75×	1.00×	1.25×	1.50×	2.00×	3.00×
p99 spread	0.1%	0.3%	**93.1%**	55.5%	28.4%	2.1%	1.1%

Scheduling policy is decisive only in a narrow band around capacity. Below capacity there is no queue, so every policy dispatches immediately. Far above it, the backlog dominates: the makespan is fixed by total work and p99 approaches it whatever order is chosen. This is invisible from any single operating point and is the most generally useful result in the study.

It also carries a practical warning: an evaluation of scheduling policies conducted at an arbitrary load may find no difference and conclude, wrongly, that policy does not matter.

4.5 EDF under overload

An earlier revision of this work asserted in three places that EDF "degrades sharply under overload", following classical theory, which warns that EDF keeps preferring already-doomed

tasks and misses them anyway while delaying tasks that were still achievable. **The data supports a weaker version of that than was asserted.** Deadline miss rate on the heavy-tailed workload:

offered load	0.50×	1.00×	1.25×	1.50×	2.00×	3.00×
fifo	0.000	0.123	0.645	0.799	0.860	0.893
priority	0.000	0.154	0.415	0.453	0.508	0.683
edf	0.000	**0.000**	**0.005**	**0.368**	**0.635**	**0.769**
wfq	0.000	0.088	0.507	0.690	0.798	0.864

EDF has the lowest miss rate of the four policies at every load up to 1.5×, and beats FIFO at every load tested: its advantage over FIFO narrows from roughly 117× at 1.25× to 1.16× at 3×, but never inverts. It is not the best policy beyond 1.5×. At 2× and 3× strict priority misses fewer deadlines — 0.508 and 0.683 against EDF's 0.635 and 0.769 — so EDF does lose ground under overload, as theory warns. What it does not do here is collapse below the baseline that reads no task metadata at all. The classical pathology in its sharp form requires deadlines that are tight relative to service time, whereas these are generous and scale with each task's own duration, so a deferred long task usually still meets its deadline. The result is therefore a property of the workload as much as of the policy.

What EDF does pay, at every load, is starvation: its longest queue wait is three to four times FIFO's and saturates near 9.6 s, because it defers precisely the large tasks whose deadlines are furthest away. Miss rate and maximum wait must be read as a pair.

4.6 The cost of centralisation

The scheduler is a deliberate single-writer component. Three measurements establish what it costs: its own capacity, the price of enforcing the single-writer invariant, and its overhead against having no scheduler at all. Section 4.7 adds the fourth cost, which is availability.

Its own capacity. With workers removed and the critical path driven directly:

stage	rate	note
dispatch, best case	261,727 tasks/s	batch 256; atomic `ZPOPMIN+XADD`
admission	15,204 tasks/s	one Redis round trip per task
end to end	**11,381 tasks/s**	both loops — the real ceiling

Dispatch is 17× faster than admission, so dispatch was never the constraint: it amortises a batch over one round trip while admission pays one per task. The scheduler can feed approximately **569 worker slots** at a 50 ms mean task. It follows that every experiment in this study ran the

scheduler at about **3% of its capacity**, so the policy comparisons measure policy rather than scheduler saturation — a validity check the work could not previously make.

These figures were re-measured after the leader lease was added and reproduce the earlier measurement of the same machine to within 1.6%.

What enforcing the invariant costs. The same measurement was run twice, once with the fencing guard inert (`--epoch 0`, election off) and once with it live:

stage	guard inert	guard live	change
admission	15,204/s	15,009/s	−1.3%
end to end	**11,381/s**	**11,325/s**	**−0.5%**
dispatch, best case	261,727/s	264,043/s	+0.9%

The correct reading is "below this measurement's noise floor", not "−0.5%": each cell is a single run, dispatch came out faster fenced than unfenced, and WFQ's admission moved the wrong way too. The structural reason the cost is small is that the fence adds one `GET` *inside* a script that has already paid for its round trip, plus a `SET` only on the first write of a new term. Enforcement rides along on work that was happening anyway, which is an argument for putting the check at the resource rather than in the client quite apart from its correctness advantages.

Its overhead, against no scheduler at all. A control arm was added in which workers consume the ingress stream directly, producing the same execution order as FIFO. 240 experiments, 960,000 tasks.

offered load	uniform	bursty	heavy_tailed	multi_tenant
0.75×	0.0%	0.2%	0.0%	0.0%
1.25×	1.5%	1.6%	1.2%	1.6%

Centralisation is unmeasurable below capacity and costs about **1.5%** above it: the extra round trip through the pending set. What that enables costs considerably more and returns considerably more (heavy-tailed, 1.25×):

arm	miss	maxwait	tasks/s	vs. none
none	0.609	2.09	272.7	—
fifo	0.644	2.29	269.4	−1.2%
priority	0.415	3.69	262.3	−3.8%
wfq	0.508	3.38	255.2	−6.4%
edf	**0.005**	8.25	244.0	−10.5%

EDF trades 10.5% throughput for a 118× reduction in deadline misses; WFQ trades 6.4% for tenant isolation. **FIFO is the one arm that fails to justify itself**: it costs 1.2% and delivers identical

ordering, so it is strictly worse than having no scheduler. FIFO earns its place here as an experimental baseline, not as a deployment choice.

4.7 What a scheduler failure costs

Section 4.6 measured the *throughput* cost of centralising scheduling. That is half the argument; the other half is availability, and an availability claim that rests on reading the code is not a measurement.

Method. `cmd/failoverbench` builds the scheduler binary, starts real processes with `os/exec` in their own process groups, and kills one with a real signal — nothing about the failure is simulated. Workers and the load generator run in the measuring process against the same Redis. The outage is defined narrowly: the gap between the last task dispatched at or before the kill and the first one after it, read from the `dispatched_at_ms` field the dispatch script stamps on every execution-stream entry. Trials in which no dispatch ever follows are **censored** rather than recorded as a large number.

The kill is jittered uniformly over one renewal interval, from a seeded generator. Without that, every trial kills at the same phase of the renewal cycle and the measured outage collapses to a single value — real, but not the distribution. The first version of this experiment used a fixed offset and produced fifteen outages within 4 ms of each other, which is what exposed the bias.

Every trial also scrapes `tq_scheduler_is_leader` from all replicas before the kill and refuses to proceed unless the sum is exactly 1, so each reported trial carries evidence that the single-writer invariant held while it ran. Across the 75 trials below it was 1 every time.

A second check reads the invariant back out of the data. Every execution-stream entry is stamped with the epoch that dispatched it, so replaying the stream in order must produce epochs that never decrease; anything else is a superseded leader that wrote after its successor. The harness counts these inversions in every arm and aborts on the first one, and the plotting script refuses to chart a run that contains any. **Across all 75 trials the count was zero.**

arm	replicas	signal	what it answers
`none`	2	—	the natural gap between dispatches, the floor everything else is read against
`graceful`	2	`SIGTERM`	what a rollout costs; the lease is released
`crash`	2	`SIGKILL`	what a crash costs; nothing is released
`single`	1	`SIGKILL`	what the lease is actually buying
`pause`	2	`SIGSTOP`, then `SIGCONT`	what the *fence* is buying; see Section 4.8

Results (15 trials per arm, 4,000 tasks each, 5 s lease, 1.5 s renewal, 1 s standby poll):

arm	n	median outage	range	tasks unfinished
`none`	15	6 ms *(largest natural gap)*	5–19 ms	0
`graceful`	15	**663 ms**	132–976 ms	0
`crash`	15	**4,362 ms**	3,536–4,951 ms	0
`pause`	15	**3,879 ms**	3,524–4,860 ms	0
`single`	15	*never recovered*	—	**37,866**

Three things are worth drawing out.

The outage is not "one lease TTL", and the difference matters. The obvious model — a crash costs a TTL — is wrong in both directions. The correct one has two terms:

```
outage = time until the lease is free + time until a standby next polls
```

A crash makes the first term `TTL − age of the last renewal`, uniform over `[TTL − renew, TTL]`. A graceful stop makes it zero, because the leader releases on the way out. Both then wait up to one `retry_interval` for a standby to notice. The predicted windows are therefore `[3.5 s, 6.0 s]` for a crash and `[0, 1.0 s]` for a rollout, and **all 15 trials in each arm fall inside them**.

The `pause` arm sits in the same window as `crash` — 15/15 inside `[3.5 s, 6.0 s]`, with a median of 3,879 ms against 4,362 ms — and it should, because up to the moment of takeover the two are the same experiment: a leader stops renewing and Redis expires its grant, whether the process is dead or merely frozen. The 483 ms difference between the medians is not interpreted here. The

two ranges overlap almost completely and n is 15; nothing in the mechanism predicts a difference, and reading one into this sample would be exactly the kind of claim Section 4.6 declined to make about a 0.5% throughput gap.

The practical consequence is that a rollout's cost is a *polling artefact*, not a property of the lease: 663 ms is simply half a poll interval. Publishing a notification on release would remove it almost entirely, which is why it appears in future work rather than as a tuning recommendation.

The floor is 6 ms, so the signal is unambiguous. The natural largest gap between dispatches under this load is a median of 6 ms and never exceeded 19 ms. A crash outage is roughly **730× that floor**, so none of these numbers depends on separating a failover from ordinary scheduling jitter.

The control arm is the argument for the lease. With one replica, no trial recovered at all. The queue stopped dispatching permanently and a median of 2,613 of 4,000 tasks per trial — 37,866 in total — never reached a terminal state. Averaging those trials as a large finite number would misrepresent them entirely, so they are censored, and the chart draws them as a hatched bar spanning the axis rather than as a value.

Nothing was executed twice. Across all 75 trials the duplicate-completion counter stayed at zero, and every trial in the four recovering arms finished all 4,000 tasks. That last property was not free; see Section 5.1.

Failover time tracks the lease, which makes it a tuning decision. The crash arm was repeated at three lease TTLs, with the renewal interval held at `TTL / 3.3` and the standby poll at 1 s throughout:

lease TTL	n	median outage	range	predicted `[TTL − renew, TTL + retry]`	inside
2 s	12	1,850 ms	1,437–1,960	[1.4 s, 3.0 s]	12/12
5 s	15	4,362 ms	3,536–4,951	[3.5 s, 6.0 s]	15/15
10 s	12	8,259 ms	7,172–9,343	[7.0 s, 11.0 s]	12/12

All 39 trials fall inside the envelope the two-term model predicts. The medians sit a little below its midpoint at every TTL, and consistently so — 0.92×, 0.87× and 0.83× the TTL respectively — which says the standby tends to find the lease sooner than a uniform poll offset would suggest. No mechanism is claimed for that; it is a small systematic offset in the right direction, and the envelope is the part the model is being tested on.

The operational reading is a trade, not an optimum. A shorter lease shortens every crash outage and gives a healthy leader fewer renewal attempts to survive a slow Redis round trip before being deposed for no reason. The 5 s default with a 1.5 s renewal allows three such attempts, which is the reason it is the default rather than the 2 s that looks better in this table.

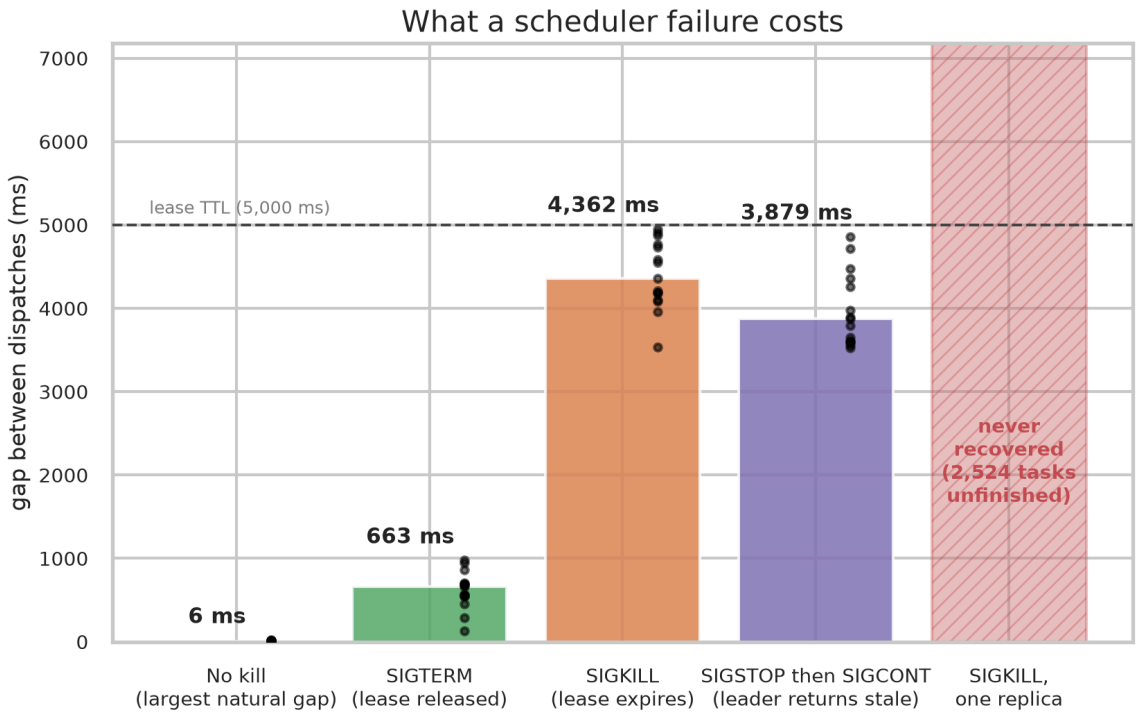


Figure 1: Failover outage by arm

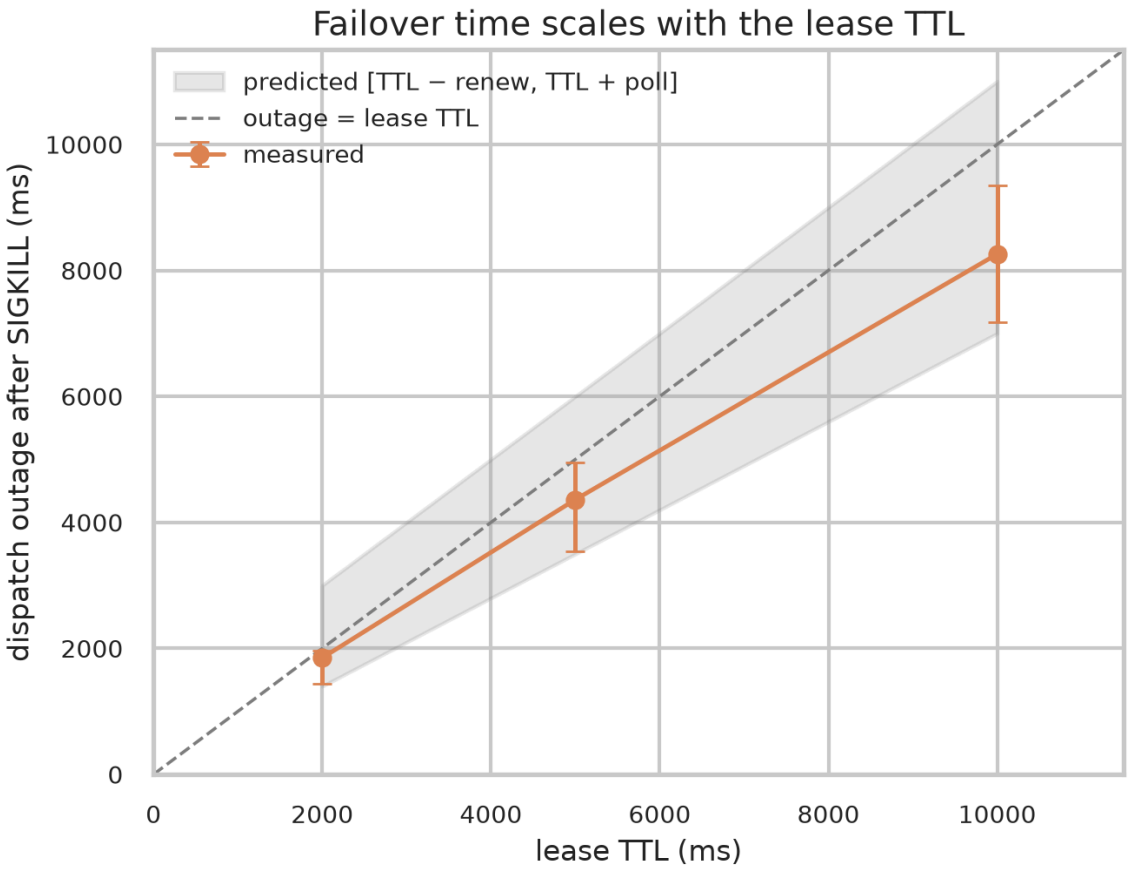


Figure 2: Failover time against lease TTL

4.8 The failure the fence is for

Everything in Section 4.7 kills the leader, and that is a weakness in the argument rather than a strength. A lease on its own would have handled every one of those arms: a dead process writes nothing, so the fencing token is never loaded. Demonstrating a safety mechanism against a failure it is not needed for proves only that it does no harm.

The case fencing tokens exist for is the one where the deposed leader is **still running**. It was stopped long enough for its lease to expire — a garbage-collection pause, a suspended container, a host that swapped, a partition that outlasted the TTL — a standby took over, and then it resumed, still holding its old epoch. Cancelling its context cannot stop it, because it was not running to observe the cancellation. Nor can it be made to check whether it still leads: any such check is separated from the write that follows it by a window in which the answer can change, which is Kleppmann's objection to using a lock alone. The check has to be at the resource, in the same atomic unit as the write.

Method. A fifth arm, `pause`, produces that state with real signals rather than a simulation of one. The leader is stopped with `SIGSTOP` for twice its lease TTL — 10.2 s measured, against a 5 s lease — and then continued with `SIGCONT`, after which it is given three further seconds to act before

the trial is allowed to end. Three conditions are verified rather than assumed, because a fault-injection experiment that does not confirm the fault occurred can pass while testing nothing:

1. the victim really is stopped, read from the operating system's process state rather than inferred from a signal call that returned no error;
2. a *different* process holds the lease before the victim is resumed, so it is genuinely superseded rather than merely slow;
3. after the resume, exactly one process still claims leadership.

Result: the invariant held, and the fence is what held it. Across 15 trials no task was dispatched under a superseded epoch, no completion was duplicated, all 4,000 tasks finished in every trial, and the leadership sum after each resume was exactly 1.

What the resumed leader ran into differs between trials, and the split is the interesting part:

what stopped the superseded leader	trials
refused by the fence, at the resource	**11 / 15**
stood down first, on its own failed lease renewal	4 / 15

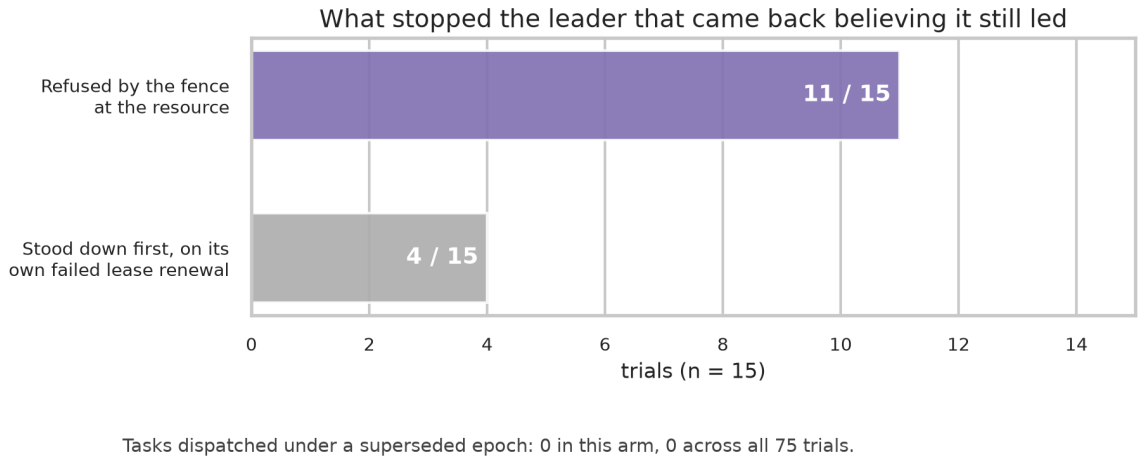
Both outcomes are correct, and the second is the same protection arriving one layer earlier. But the first group is the one that matters for the design question: **in 11 of 15 trials the resumed leader reached Redis with a write before its own lease machinery had noticed anything was wrong.** A deployment with leases but no fencing would have admitted a second writer in those eleven cases. The logs show how narrow the margin is — resume, lease loss and fence refusal all land within the same millisecond:

```
22:53:55.978  resumed the superseded leader   owner=sched-1 successor=sched-0
22:53:55.978  lost the scheduler lease        owner=sched-1 epoch=1
22:53:55.978  fenced out by a newer epoch, standing down   operation=dispatch
```

That is not a margin any implementation should be asked to win by being quick.

The claim is falsifiable, and was falsified on purpose. The corresponding integration test drives an engine into the same state deterministically, by giving it a lease whose renewal interval cannot fire inside the test window, so the fence is the only thing that can stop it. With the fence removed from the Lua guard, that test fails immediately: **172 of 400 tasks were dispatched under a dead term**, concurrently with the legitimate leader, and the dispatch log records every one of them. A test that has never been observed to fail is not evidence about the mechanism it guards.

What this does not cover. A stopped process resumes with its connection to Redis intact. A network partition is the other half of this failure mode and is not reproduced here: it would break the path to the store as well as the process's view of time, which is a different experiment and is named in Section 6.1 rather than approximated.



Tasks dispatched under a superseded epoch: 0 in this arm, 0 across all 75 trials.

Figure 3: What stopped the superseded leader

4.9 What the fence rests on

Sections 4.7 and 4.8 measure the scheduler failing. Both of them rest on an assumption about the *store* that was stated plainly and never tested: that Redis does not lose acknowledged writes. The fencing token is a monotonic counter held in Redis, and its guarantee — once epoch N has written, nothing below N writes again — is exactly as durable as that counter. This section tests the assumption instead of restating it.

Method. `cmd/storefaultbench` builds a real Redis Sentinel cluster out of local `redis-server` and `redis-sentinel` processes — one master, two replicas, three sentinels — points the queue at it through Sentinel discovery, and then breaks the store underneath a running workload. Four arms, 10 trials each:

arm	fault	mitigation
`clean`	master killed with replicas caught up	none
`lagging`	replication cut, then the master killed	none
`epochwait`	the same	the lease waits for a replica to acknowledge a new epoch
`guarded`	the same	the above, plus `min-replicas-to-write` on the master

Three details decide whether this experiment measures anything, and each was found by getting it wrong first.

A stopped process is not a partitioned one. The obvious way to make a replica fall behind is `SIGSTOP`. It does not work: the kernel goes on accepting and acknowledging TCP segments for a stopped

process, so the replication stream accumulates in the socket buffer and is applied in full on resume. The first version of this arm reported no data loss for that reason. Losing data in transit requires cutting the path, so replication now runs through a small proxy (`internal/netfault`) that can be severed.

A graceful shutdown is not a crash. Redis 7 and later wait for replicas to catch up during `SHUTDOWN`, so a politely stopped master loses nothing. Every kill here is `SIGKILL`.

Sentinel repairs the fault. Sentinel reconfigures a replica whose master address it does not recognise, which silently undoes the injected partition on its own refresh period. The fault window is therefore short and fixed, and the harness **verifies that replication is still down at the moment the master dies**, aborting the trial otherwise. A fault-injection experiment that does not confirm the fault survived to the moment it mattered can pass while testing nothing.

Results (10 trials per arm, 2,000 tasks each, 2 s lease):

arm	one token, two leaders	acknowledged tasks destroyed	submissions accepted	completed
`clean`	**0 / 10**	0	20,011	20,011
`lagging`	**10 / 10**	**10,010**	20,020	10,010
`epochwait`	**10 / 10**	10,009	20,019	10,010
`guarded`	**0 / 10**	0	10,010	10,010

The fencing token is not safe under a Redis failover. In every unguarded trial the same token was handed to two different leaders:

```
a fencing token was issued twice to different leaders
epoch=2  first_holder=sched-2  second_holder=sched-1
```

A token that identifies two terms excludes neither. The mechanism is straightforward once seen: the epoch counter is advanced by an `INCR` on the master, the master dies before that reaches a replica, and the promoted node hands the same number out again. Around 1,000 tasks per trial went with it — work the producer had been told was accepted, which then existed nowhere.

The obvious mitigation does not work, and the reason is instructive. Making the lease wait for a replica to acknowledge the new epoch before acting on it (`epochwait`) failed in 10 trials out of 10, exactly as often as no mitigation at all. The counter is incremented *before* the wait, so a master that dies in between has already lost it; the wait can report that the write did not survive, but it cannot un-issue the token. Detecting a lost write is not the same as preventing one. This is the arm worth keeping in the report precisely because it is the fix a reader would reach for first.

What works is refusing the write. With `min-replicas-to-write` on the master, the `INCR` itself fails while no replica can receive it — `NOREPLICAS Not enough good replicas to write` — so no token is issued that could be lost, and no task is accepted that cannot be kept. Zero violations in 10 trials, zero tasks destroyed.

The cost is availability, and it is visible rather than hidden. The guarded arm accepted 10,010 submissions where the unguarded arms accepted 20,020. That gap is not lost throughput: the unguarded arms completed 10,010 tasks as well. The difference is that they *also told the producer* they had accepted another 10,010 which they then destroyed. The guarded configuration converts silent data loss into an error the producer can see and retry. During normal operation the cost is nothing measurable: `min-replicas-to-write` is a local check on the master's replica count, and the epoch wait happens once per leadership term rather than once per write.

A note on which number to trust. The `epoch_rewound` column disagrees with `token reused` — 0/10 in `lagging` against 10/10 in `epochwait` — and the reused-token count is the reliable one. Rewinding is detected by reading the counter from the promoted node after the fact, and by then a new leader has often already advanced it past where it was, hiding the gap. Token reuse is observed continuously and recorded in the harness's own memory, outside the store. That design choice was deliberate: asking the survivor of an accident to describe the part it slept through is not a measurement.

Scope. This is Redis Sentinel with asynchronous replication, which is what the shipped deployment uses. It is not a claim about Redis Cluster, about Redis with AOF `appendfsync always`, or about any store with real consensus. The finding is narrower and more useful than "Redis loses data": a fencing token inherits the durability of whatever holds it, and the default configuration of the store this system ships with does not provide enough of it.



Figure 4: What a Redis failover does to the fence

4.10 An outside reference point

Every throughput figure so far is internal: this scheduler against that scheduler, this policy against that policy, all inside one codebase on one machine. That answers which choice is better *here* and says nothing about whether here is any good. Section 4.6 measured the cost of centralising the scheduling decision against a control that simply switched the scheduler off, which is a control written by the same author as the thing it controls for.

Method. Asynq is the comparison, chosen for a structural reason rather than for popularity: it is Go, it is backed by Redis, and its workers pull tasks directly from Redis lists with no central scheduler at all. The difference between the two systems is therefore close to the single design

decision this project exists to examine. It runs as a separate Go module (`bench/asynq`) so that the dependency never enters the shipped library's graph.

Fairness is the only real difficulty in a benchmark like this, so the controls are stated rather than assumed: the same Redis server, one system after the other and never concurrently; the same task count, entirely pre-enqueued before any worker starts, so the measurement is drain time and no producer rate sits in the way; the same sleep as simulated work; the same total concurrency; and the order of the two systems alternated across repetitions so neither is systematically first. Latency is computed by the harness from the instant the workers are released, identically on both sides, rather than read from either system's own instrumentation.

The comparison uses this project's FIFO policy with leader election off, which is the configuration closest to what Asynq provides. The cost of the fence and the lease is measured separately in Sections 4.6 and 4.7.

Results (5 repetitions per point, 8,000 tasks each, 32 slots):

task duration	this queue	Asynq	difference	slot ceiling	this queue	Asynq
0 ms	8,075/s	**12,478/s**	**−35.3%**	—	—	—
1 ms	8,265/s	**11,896/s**	**−30.5%**	32,000/s	25.8%	37.2%
2 ms	8,611/s	**12,181/s**	**−29.3%**	16,000/s	53.8%	76.1%
5 ms	**5,810/s**	5,522/s	+5.2%	6,400/s	90.8%	86.3%
10 ms	**2,983/s**	2,805/s	+6.3%	3,200/s	93.2%	87.7%
20 ms	**1,492/s**	1,465/s	+1.8%	1,600/s	93.3%	91.6%

Below about three milliseconds per task, centralising the decision costs roughly 30% of throughput. That is the price of this architecture, measured against somebody else's production implementation of the alternative rather than against an internal control, and it is a larger number than Section 4.6's 1.5%. The two are not in conflict: 4.6 compares against the same codebase with the scheduler removed, which still routes every task through the same Redis structures, while Asynq's workers pull from a list with no dispatch step at all. The comparison here is against a different architecture, not a disabled feature, and the gap is correspondingly wider.

Above about five milliseconds the difference disappears, and slightly reverses. Both systems then sit within a few per cent of each other and at 86–93% of the ceiling that slot occupancy imposes, which is another way of saying the benchmark has stopped measuring queue machinery and started measuring `sleep`. This queue is marginally ahead there, and consistently closer to the ceiling, which is the sort of small difference this report has elsewhere declined to interpret — a plausible mechanism is that batched dispatch keeps slots filled more evenly than independent per-worker polling, but that has not been tested and is offered as a hypothesis, not a finding.

What this means for the design. The crossover sits between 2 ms and 5 ms on this machine. A workload of tasks shorter than that should not use a central scheduler, and this project's own results say so: Section 4.1 found that scheduling policy is decisive only in a narrow band around capacity, and a sub-millisecond task queue is not where a global policy earns its cost. Where tasks take milliseconds or longer — which covers most of what task queues are actually used for — centralisation is free in throughput terms, and everything Sections 4.2 to 4.5 measured is what it buys.

What is not claimed. Asynq does things this project does not: scheduled and recurring tasks, task groups, a web UI, and years of production use. This project does things Asynq does not: a pluggable global policy with deadline-aware and fair-queuing disciplines. This is one axis, measured carefully, on one machine.

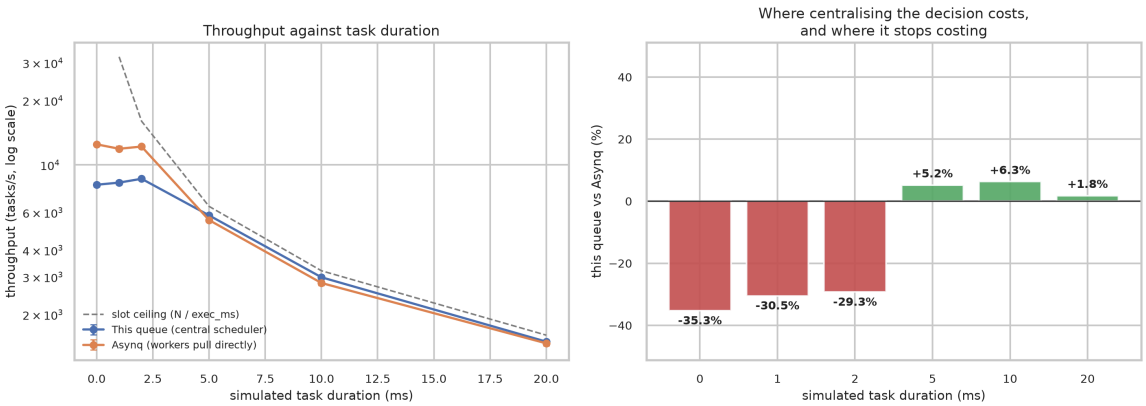


Figure 5: This queue against Asynq

4.11 A scheduler cut off from a Redis that stays up

Sections 4.7 to 4.9 leave one fault shape untested, and it is the one in which the leader knows the most and can do the least. A killed leader writes nothing. A frozen one, in 4.8, observes nothing, and resumes with its connection intact. A leader partitioned from a healthy Redis keeps running, watches every call fail, and still cannot tell whether the store is merely unreachable or it has already been replaced. Those two situations call for opposite responses. Standing down on the first failed round trip turns every transient blip into a failover; carrying on regardless is how two schedulers end up dispatching. This queue does neither: the lease keeps retrying, and the fence makes the ambiguity harmless if it turns out the leader was replaced.

Method. `cmd/partitionbench` starts two real scheduler processes, as 4.7 does, but each reaches Redis through its own proxy (`internal/netfault`), the one built for 4.9. Cutting a proxy closes its open connections and refuses new ones, so the leader's path is severed at the socket while Redis, the workers, the harness and the standby all keep theirs. The cut lands 3 s into the trial plus a seeded jitter of up to one renewal interval, for the reason given in 4.7. After the path is repaired the replica that was cut off is given 4 s to act before the harness reads the lease holder, scrapes the fence-refusal counter from that replica, and checks that exactly one replica claims leadership. The dispatch log is replayed for epoch inversions as in every failover run. Load, lease and poll settings

are those of 4.7: 4,000 tasks at 400/s, 20 ms each, 32 worker slots, a 5 s lease renewed every 1.5 s, a 1 s standby poll.

arm	partition	what it answers
`none`	—	the natural gap between dispatches
`brief`	half the lease, 2.5 s	does a blip shorter than the lease cost a failover? It should not
`long`	twice the lease, 10 s	is the leader replaced, and refused when it comes back?
`flapping`	700 ms cut, 700 ms healed, for 3 × the lease	does a path that cannot settle cause a split brain, or a storm of leadership changes?

The host is watched, not trusted. Two earlier attempts at this experiment were run on battery power, and macOS slept the machine partway through. A suspended host freezes both replicas mid-handover; they wake with stale gauges that scrape as two leaders, and a gap in the dispatch record that reads as an outage. Each trial therefore measures the host's own suspension directly (`internal/hostclock`), and marks any trial in which the machine slept as suspect. Suspect trials are dropped and counted, never averaged. Both runs reported here were made on mains power under `caffeinate`, and **no trial in either was suspect**. The earlier attempts are discarded and not counted.

Results (12 trials per arm, 48 in all):

arm	n	partition	median outage	range	leadership moved	refused by the fence on heal
`none`	12	—	5 ms *(largest natural gap)*	5–6 ms	0 / 12	—
`brief`	12	2,500 ms	**2,526 ms**	2,409–2,624 ms	**0 / 12**	0 / 12
`long`	12	10,000 ms	**4,376 ms**	3,537–4,955 ms	**12 / 12**	**11 / 12**
`flapping`	12	15,410 ms, 11 cuts	774 ms	442–808 ms	7 / 12	7 / 12

In every one of the 36 partitioned trials exactly one replica claimed leadership once the path was repaired. Across all 48, no task was dispatched under a superseded epoch, none was completed twice, and every trial finished all 4,000 tasks.

A partition shorter than the lease costs its own length, and nothing else. The `brief` arm's outage is the partition: a median of 2,526 ms against a cut of 2,500 ms. Leadership did not move in any of the twelve trials. The leader's renewals failed for two and a half seconds, it kept its grant because the grant had not expired, and it resumed dispatching the moment it could. That is the behaviour the retry policy exists for. A lease that stood down on its first failed renewal would have turned each of these trials into a failover and cost a crash's worth of outage, about 4.4 s, for a 2.5 s blip.

A partition longer than the lease costs a crash, not the partition. In the `long` arm the path was down for 10 s and dispatch resumed after a median of 4,376 ms, because the standby took over in all twelve trials. **All twelve outages fall inside the `[3.5 s, 6.0 s]` window Section 4.7′s two-term model predicts for a crash**, and the distribution sits where the crash arm's did (median 4,362 ms there). From the queue's point of view a partitioned leader whose lease expires is a dead one: it stopped renewing, and Redis does not distinguish why. The remaining 5.6 s of partition cost nothing, because by then somebody else was dispatching.

The fence is what refused the leader that came back. When the path was repaired, the old leader was still running and still believed it led. In **11 of 12 trials** it reached Redis with a write under its old epoch before its own lease machinery had concluded anything, and Redis refused it; in the twelfth its lease renewal failed first and it stood down without trying. This is the same split Section 4.8 found for a frozen leader, 11 of 15, and it carries the same reading: a deployment with a lease but no fence would have admitted a second writer in eleven of these twelve trials. The difference from 4.8 is that a partitioned leader is not asleep. It spent the partition awake, watching its renewals fail, and still reached Redis with a stale write before it concluded anything. Being awake does not help a process answer a question it has no way to answer from inside.

A path that will not settle does not cause a storm. The `flapping` arm cut the path eleven times in 15.4 s. Leadership moved in 7 of 12 trials and stayed put in the other 5, and in no trial did it move more than once: the epoch counter ended at 2 or at 1 every time. Every leader that was replaced was refused by the fence when it next reached Redis, 7 of 7. The largest dispatch gap anywhere in the arm was 1,720 ms: dispatch carried on through each healed half-period, and a handover, when one happened, did not open a gap of its own longer than that. Whether a given trial fails over depends on how the 700 ms flap happens to line up with the 1.5 s renewal, so 7 of 12 is a property of these two periods and is not offered as a rate.

The lease, not the partition, bounds what a partition costs. The named arms test one length either side of the lease. A second run sweeps seven lengths, from a quarter of the lease to three times it, 8 trials each, with everything else unchanged:

partition	n	median outage	range	leadership moved	refused by the fence on heal
1,250 ms (0.25×)	8	1,307 ms	1,227–1,414 ms	0 / 8	0 / 8
2,500 ms (0.5×)	8	2,546 ms	2,369–2,613 ms	0 / 8	0 / 8
3,750 ms (0.75×)	8	3,743 ms	3,703–3,864 ms	2 / 8	2 / 8
5,000 ms (1×)	8	4,378 ms	3,697–4,955 ms	8 / 8	8 / 8
7,500 ms (1.5×)	8	4,328 ms	3,529–4,906 ms	8 / 8	7 / 8
10,000 ms (2×)	8	4,465 ms	4,181–4,733 ms	8 / 8	6 / 8
15,000 ms (3×)	8	4,131 ms	3,527–4,887 ms	8 / 8	5 / 8

Up to three quarters of the lease the outage is the partition, to within a few per cent. From one lease upward it stops growing: a 15 s partition cost a median of 4,131 ms, less than the 5 s one, and **all 32 trials at or beyond the lease fall inside the crash window**. Where leadership moves is predicted, not merely observed. At the cut the grant has between `TTL − renew` and `TTL` left, 3.5 s to 5 s, depending on how long ago it was last renewed. A partition shorter than 3.5 s therefore cannot cost a failover (0 of 16), one of 5 s or more always does (32 of 32), and one in between does or does not according to where in the renewal cycle it landed: 2 of 8 at 3.75 s. The cost of a partition of any length is capped by the lease, which makes the lease TTL the single setting that decides it.

One pattern is reported without being interpreted. The share of returning leaders refused by the fence, rather than stopped by their own failed renewal, falls as the partition lengthens: 8 of 8 at one lease, then 7, 6 and 5 of 8. The lease keeps no clock of its own (a cut-off leader goes on believing it leads until a renewal reaches Redis and is told otherwise), so on every heal the renewal ticker and the dispatch loop race to be first, and something about a long-severed connection evidently shifts that race. What, is not tested, and at eight trials per length the trend itself is weakly supported. Both outcomes are safe: across all 56 sweep trials there was exactly one leader after every heal, no epoch inversion, no duplicate completion and no unfinished task, and none was suspect.

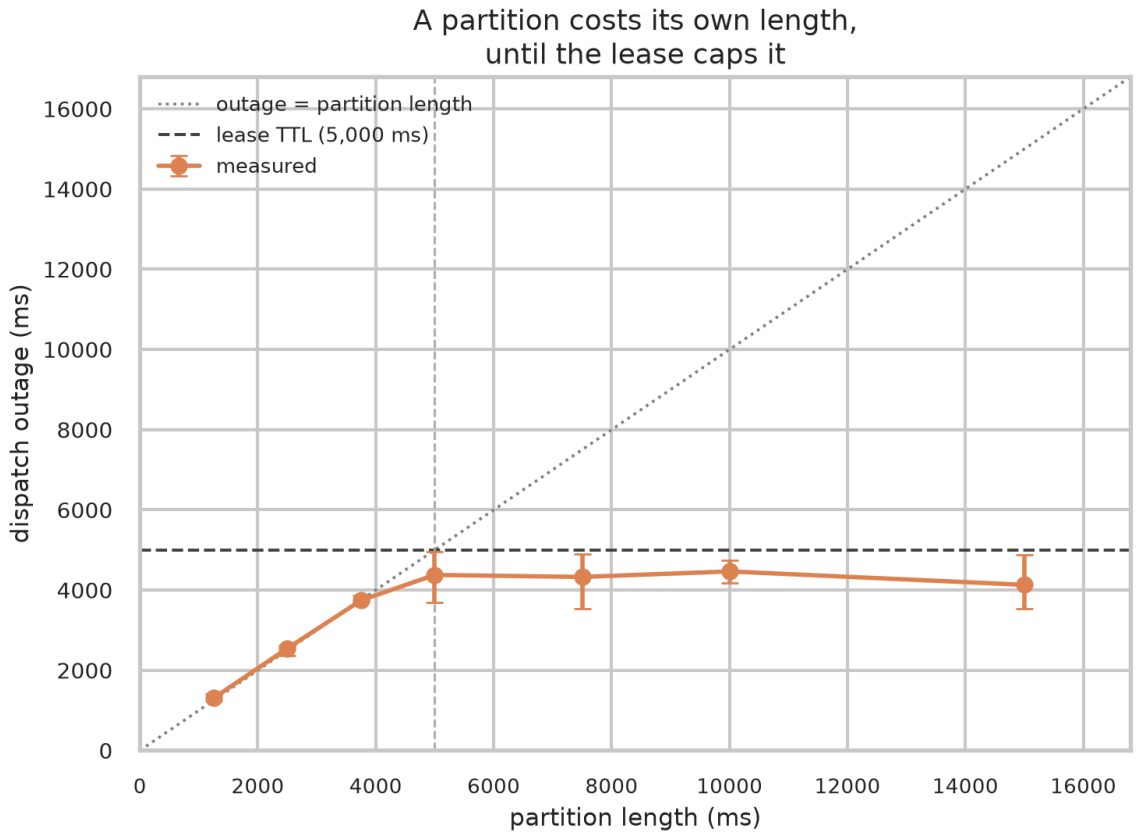


Figure 6: A partition costs its own length until the lease caps it

What is not claimed. Every process ran on one host, and the partition is a severed socket rather than a real network: this tests how the lease and the fence respond to a lost path, not how long a real network takes to fail or how a real TCP stack notices. Only the leader's path to Redis is cut. Workers are never partitioned, and neither is the standby; a partition that separated workers from the store, or one that left the leader able to reach Redis but not its workers, is a different experiment. And the fence's guarantee here is exactly Section 4.9's: it holds because this Redis does not fail over. Combine this partition with a store failover and the two results compose, in the direction 4.9 describes.

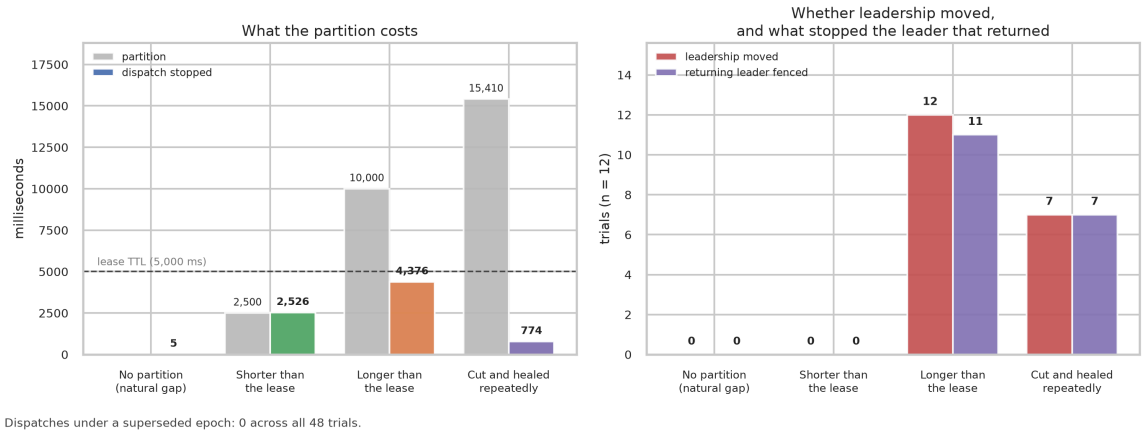


Figure 7: What a partition costs

5. Threats to validity

5.1 Corrections made during this study

Three quantitative claims were found to be wrong after more data was collected, two defects were found by the failover experiment rather than by review, and four documentation errors were found later by recomputing every number this report quotes from the raw data.

The bursty load confound (Section 3.2): bursty experiments ran at 1.57× capacity while others sat at 1.25×, because the workload ignores its nominal arrival-rate parameter. Corrected by deriving arrival rate per workload from measured capacity.

EDF's overload behaviour (Section 4.5): asserted from theory in three places before measurement, and contradicted by the sweep. That correction itself overreached, and is corrected again below.

EDF's real-work degradation (Section 5.3): first reported as 27× from three repetitions on the one workload already shown to require twenty-five. At n=25 the correct figure is 8.4×. The direction held; the magnitude was inflated more than threefold, and the sleep-mode value it was measured against was a lucky draw — 0.099 ± 0.183 at n=25, a standard deviation larger than its own mean.

Two defects the failover experiment exposed (Section 4.7). Neither was visible to testing or to reading the code.

The first was a durability bug. Every killed-leader trial finished exactly one task short of the 4,000 submitted, which is a small enough discrepancy to be dismissed as noise and was not: `XREADGROUP` with `>` only delivers entries nobody has seen, so an ingress entry the dead leader read and never acknowledged belonged to a consumer name that would never return. The task had been accepted from the producer and was then never scheduled — not pending, not in flight, not dead-lettered, and counted by nothing. It was fixed with an `XAUTOCLAIM` pass over the ingress group, and the regression test was confirmed to fail with that pass removed.

The second was a deadlock in the lease's campaign loop, on the one path that neither the integration tests nor an ordinary shutdown takes: a term that ends *by itself* while the lease is still healthy, which is exactly what a fenced leader does. The term's result was collected once inside a select and then waited for a second time, on a channel that had already been drained. A regression test that runs three consecutive self-ending terms reproduces it (one term instead of three) and now guards it.

Both are worth reporting for the same reason as the quantitative corrections: the experiment earned its place by finding them, not only by producing a number.

Four errors found by recomputing the report from its data (October 2026). `scripts/export_site_data.py`, written to feed the project's results site, recomputes the numbers quoted here from the raw CSVs and refuses to publish if any disagree. Its first run found four errors that review had not:

1. *The bursty rows of the main matrix were not at 1.25× load.* Section 3.2 identified the bursty confound and the load sweep corrected it, but the 25-repetition main matrix was never re-run, and Section 4.1 described all sixteen of its cells as 1.25×. Its bursty rows are at about 1.57×; Section 4.1 now says so.
2. *EDF was not the best policy at every load.* Section 4.5 claimed the lowest miss rate "at every load tested" beside a table showing strict priority lower at 2× and 3×. The correct statement is narrower: best up to 1.5×, and better than FIFO everywhere.
3. *Two ratios were computed from rounded values.* "122×" (Section 4.6) and "129×" (Section 4.5) divide a miss rate printed as 0.005 into another; the unrounded rates give 118× and 117×. Rounding a small denominator to one significant figure moves the quotient by several per cent.
4. *The store-failure experiment used 2,000 tasks per trial, not 3,000.* The harness default is 2,000 and every clean trial shows it submitted that many; the stated figure was wrong, and with it Appendix B's task total, which falls from 5,476,000 to 5,436,000.

None of the four changes a conclusion's direction. All four are the kind of error a hand-copied number accumulates and a recomputed one cannot.

5.2 Simulated execution

Workers sleep for a task's declared duration by default. This makes task sizes exactly controllable and removes application variance, at the cost that a sleeping worker consumes no CPU: N slots always deliver N-way parallelism regardless of core count, and utilisation measures slot occupancy rather than work.

5.3 The sleep model is not neutral

To test whether the conclusions survive real work, an execution mode was added that burns each task's duration in chained SHA-256 — chosen because it is data-dependent and cannot be strength-reduced away — with iterations calibrated per machine. The same experiment was run both ways at 8 slots on 10 cores, to keep capacity slot-bound rather than core-bound.

Measured execution inflates by 1.5–1.9× even at 8 slots on 10 cores, because worker goroutines share the machine with Redis and the Go runtime. Deadline miss rate on the heavy-tailed workload, 25 repetitions:

sched	sleep	cpu	degrade	inflation	p99 ratio
edf	0.099 ± 0.183	0.829 ± 0.173	**8.4×**	1.88×	4.92×
priority	0.408 ± 0.046	0.643 ± 0.078	1.6×	1.85×	3.19×
wfq	0.474 ± 0.147	0.851 ± 0.039	1.8×	1.69×	3.07×
fifo	0.675 ± 0.130	0.866 ± 0.048	1.3×	1.54×	3.41×

Under real CPU, **strict priority overtakes EDF** (0.643 against 0.829, separable at n=25) and EDF's advantage over FIFO nearly vanishes. EDF shows both the largest execution inflation and the worst p99 degradation of any policy.

WFQ, by contrast, is unaffected. Per-tenant p95 latency under skew:

exec mode	small p95, fifo	wfq	advantage
sleep	2.750 s	0.086 s	31.8×
cpu	5.299 s	0.133 s	**39.9×**

The opposite was predicted, on the grounds that WFQ computes virtual finish times from declared cost exactly as EDF computes ordering from declared deadlines. The reason it does not degrade is the more interesting finding:

> **WFQ's use of cost is relative; EDF's is absolute.** WFQ compares tenants against one another, so a cost error inflating every task by roughly the same factor is common-mode and cancels in the ratio. EDF compares each task against a wall-clock deadline, where the same error does not cancel. A cost model can be badly wrong and still produce correct fair-queuing decisions; it cannot produce correct deadline decisions.

FIFO degrades least of all, for an instructive reason: it reads no task metadata whatsoever, so nothing about it depends on the duration model being accurate. Under a wrong cost model, knowing nothing is a form of robustness.

The practical consequence is that the EDF results in Sections 4.1 and 4.5 should be read as an upper bound attainable only with perfect cost information. The WFQ fairness results are not subject to that caveat: they were re-tested under real work and improved.

5.4 Remaining limitations

1. **Redis Sentinel, not Redis Cluster.** Section 4.9 tests a Sentinel failover and finds that the fencing argument does depend on store durability, with a configuration that restores it. Redis

Cluster is still out of scope, and the Lua scripts, while declaring all their keys, have not been validated against its key-slot constraints.

2. **One scheduler dispatches at a time.** Replicas buy availability, not throughput. Scheduling capacity is still bounded by one process, which bounds the scale at which these results apply.
3. **At-least-once only.** Duplicate execution is expected; only completion recording is idempotent.
4. **Fault injection covers one path at a time, on one host.** Worker crashes, application errors, scheduler kills, a frozen-then-resumed scheduler, a Redis failover, a severed replication link and a scheduler partitioned from a healthy Redis (Section 4.11) are all injected, but separately. A partition that coincides with a store failover, a partition between workers and the store, and an asymmetric partition are not tested, and every partition here is a socket closed by a local proxy rather than a real network failing.
5. **One external baseline, on one axis.** Section 4.10 compares against Asynq, which shares this project's language and store and differs mainly in having no central scheduler. There is no comparison against Celery, Sidekiq or Temporal, and no comparison of features rather than throughput.
6. **Timing is not reproducible**, only the workload. Cross-machine comparison is not supported.
7. **The real-work comparison is narrow**: three of four workloads, a single offered load, one machine's core count.
8. **Failover is measured beside the load, not under it.** Every trial kills the leader while the queue is in steady state, so the reported outage is the time to resume dispatching, not the time to work off the backlog that the outage created.
9. **All replicas share one host.** Co-locating them removes the network from the failover path, so these outages are a lower bound on what a real multi-node deployment would see.

6. Conclusions

Scheduling policy matters in a narrow band. The spread between best and worst policy moves from 0.1% below capacity to 93% at capacity and back to 1.1% at 3× capacity. Evaluations conducted at a single arbitrary load risk concluding that policy is irrelevant.

Centralised scheduling is cheap; using it is not, and it is worth it. Centralisation costs about 1.5% of throughput. Acting on it costs 6–10% and returns a 118× reduction in deadline misses (EDF) or 35× lower latency for small tenants (WFQ). FIFO alone does not repay its own overhead.

Availability is the other half of the centralisation argument, and it is bounded. Enforcing the single-writer invariant costs under 1% of scheduling throughput, because the fence rides inside a script that has already paid for its round trip. A crash costs a median 4.4 s of dispatch outage against a 5 s lease; a rollout costs 0.66 s, and that figure is a polling artefact rather than a property of the lease. With no standby, the queue does not recover: a median of 2,613 of 4,000 tasks per trial were left permanently unscheduled. The single-writer scheduler is defensible, but only with a second replica behind it.

A lease is not enough, and the gap is measurable rather than theoretical. When the leader is frozen past its lease and then resumed instead of killed, it comes back still believing it leads. In 11 of 15 such trials it reached Redis with a write before its own lease machinery had noticed

anything was wrong, and what refused it was the epoch check at the resource. Removing that check and repeating the deterministic version of the experiment puts 172 of 400 tasks on the wire under a dead term. The distinction between holding a lock and proving at the point of the write that you still hold it is the difference between those two outcomes.

A fencing token is only as durable as the store that holds it. Under a Redis Sentinel failover to a replica that had not caught up, the same token was issued to two different leaders in 10 trials out of 10, and about a thousand acknowledged tasks per trial were destroyed. The mitigation a reader reaches for first — having the leader wait for its epoch to replicate — fails just as often, because the counter is advanced before the wait and a lost write cannot be un-issued. What works is configuring the master to refuse writes it cannot replicate, which converts silent loss into an error the producer can see. The correctness of a fencing scheme is a property of the token *and* its storage, and only one of those is in the scheduler's source code.

Centralisation is free for real tasks and expensive for trivial ones. Against Asynq — same language, same store, no central scheduler — this queue is about 30% slower when tasks take under three milliseconds, and within a few per cent once they take five or more, where both sit at 86–93% of the ceiling slot occupancy imposes. The crossover is the practical boundary of this design: a sub-millisecond workload should not route every task through one process, and that is exactly the regime where Section 4.1 found policy makes no difference anyway.

Metric choice can hide the effect entirely. Jain's index over completed service reports 0.246 for every policy under tenant skew, because it is demand-limited. The 35× isolation WFQ provides is visible only in per-tenant latency.

The sleep model systematically flatters deadline scheduling. EDF's advantage is contingent on cost information that real execution invalidates; WFQ's is not, because its cost errors are common-mode. Any study of deadline-aware scheduling that simulates execution with sleeps should be read with this in mind.

Diagnostics can invert. Confidence-interval coverage, the obvious test for whether enough repetitions have been run, ranked the most-converged workload worst and the least-converged workload well. The magnitude of shift in the point estimate is the diagnostic that works.

6.1 Future work

Cost *estimation* from observed durations rather than declared ones is the natural next step, and Section 5.3 predicts it would recover much of EDF's lost advantage while leaving WFQ unchanged. Beyond that: ageing or lottery scheduling as a starvation-free alternative to strict priority; WF2Q or deficit round robin to quantify what self-clocking costs; a tenant-sharded scheduler to lift the single-writer bound; and latency-aware autoscaling driven by queue depth rather than CPU.

Sections 4.7 to 4.11 add two of their own. **Composing the faults**: Section 4.11 partitions a scheduler from a Redis that stays up and 4.9 fails Redis over under a scheduler that stays connected, but no experiment yet does both at once, or partitions workers rather than schedulers. The harnesses for both already share the proxy that would do it. And **a release notification**: a graceful handover

currently costs up to one standby poll interval, which is a polling artefact rather than anything fundamental, and publishing on release would remove it.

Appendix A: Reproducing these results

```
make redis-up
make bench-full REPETITIONS=25 RUN=main            # Section 4.1
make sweep RUN=sweep-loads REPETITIONS=3           # Section 4.4
go run ./cmd/benchmark --schedulers none,fifo,priority,edf,wfq \
  --loads 0.75,1.0,1.25,1.5 --repetitions 3 --run-id control-arm
go run ./cmd/benchmark --config experiments/realwork.yaml \
  --exec-mode cpu --repetitions 25 --run-id realwork    # Section 5.3

# Section 4.6: the scheduler's own ceiling, and what the fence costs.
go run ./cmd/schedbench --tasks 20000 --epoch 0 --csv results/ceiling-unfenced/
ceiling.csv
go run ./cmd/schedbench --tasks 20000 --epoch 1 --csv results/ceiling-fenced/
ceiling.csv

# Sections 4.7 and 4.8: failover, the gray failure, and sensitivity to the TTL.
# The `pause` arm of this run is Section 4.8; it needs no separate command.
make failover FAILOVER_REPETITIONS=15 RUN=failover
go run ./cmd/failoverbench --arms crash --repetitions 12 --seed 2 \
  --lease-ttl 2s  --lease-renew 600ms --run-id failover-ttl2s
go run ./cmd/failoverbench --arms crash --repetitions 12 --seed 3 \
  --lease-ttl 10s --lease-renew 3s    --run-id failover-ttl10s

# Section 4.9: what happens to the fence when Redis itself fails over.
# Builds its own throwaway Sentinel cluster; needs redis-server and
# redis-sentinel on PATH and touches no other Redis.
make storefault STOREFAULT_REPETITIONS=10 RUN=storefault

# Section 4.10: the external baseline. A separate Go module, so Asynq never
# enters the shipped library's dependency graph.
make baseline BASELINE_REPETITIONS=5 RUN=baseline

# Section 4.11: a scheduler partitioned from a Redis that stays up, by arm and
# against partition length. Timing runs: mains power, idle machine.
make partition PARTITION_REPETITIONS=12 RUN=partition
make partition-sweep RUN=partition-sweep

make plots RUN=main && make sweep-plots RUN=sweep-loads
make failover-plots RUN=failover
make storefault-plots RUN=storefault
make baseline-plots RUN=baseline
make partition-plots RUN=partition SWEEP_RUN=partition-sweep
```

The failover and partition runs start real scheduler processes and kill or isolate them, so they need a machine that is otherwise idle: the outage is a timing measurement, and anything else

competing for CPU shows up in it. On a laptop they also need mains power. Both harnesses detect a host that slept and mark the trial suspect, but a run on battery can lose most of its trials that way.

Source, configurations and full result schemas: https://github.com/anishc23/distributed-task-queue-scheduler

Every result in this report, as interactive charts that name the files they were computed from: https://anishc23.github.io/distributed-task-queue-scheduler/

Appendix B: Experiment inventory

run	experiments	tasks
Main matrix, 25 repetitions	400	1,600,000
Offered-load sweep	336	1,344,000
No-scheduler control	240	960,000
Real work, heavy-tailed, 25 reps	200	400,000
Real work, initial comparison	48	96,000
Real work, tenant isolation	40	80,000
Scheduler failover, five arms	75	300,000
Failover, lease-TTL sensitivity	24	96,000
Store failure under Sentinel, four arms	40	80,000
External baseline against Asynq	60	480,000
Scheduler partition, four arms	48	192,000
Partition-length sweep	56	224,000
Total	**1,567**	**5,852,000**

Failover rows count tasks *submitted*: the single-replica control arm leaves a median of 2,613 per trial permanently unscheduled, which is the result rather than a defect of the accounting. Two further failover runs were discarded for the reasons given in Section 3.4 and are not counted here, nor are the earlier partition attempts that Section 4.11 describes, which a sleeping host contaminated. The scheduler-ceiling measurements of Section 4.6 are also excluded, since they drive the critical path directly rather than running experiments.